

Lenguaje PL/SQL Oracle – Parte 2

Más sobre Declaración de Variables

Otra característica elegante de PL/SQL es la capacidad de anclar el tipo de dato de una variable a una columna de una tabla mediante %TYPE. Hay dos atributos útiles al momento de declarar variables en sintonía con campos y/o registros de una tabla: %TYPE y %ROWTYPE.

Cuando se declara una variable para que contenga el contenido de una columna determinada de una tabla, la variable debe ser del mismo tipo y tamaño que la columna a la que se hace referencia. Para evitar que tener que estar buscando el tipo que coincida, y si éste cambiara en la tabla cambiar el código en los procedimientos se puede usar **%type**, esto hace que la variable tenga el mismo tipo de datos que la columna de la tabla.

```
variable tabla.columna%TYPE
```

%TYPE es un atributo que permite declarar una variable de tipo campo que refleja la estructura de un campo específico de una tabla determinada. En este caso l_apellido es del mismo tipo de campo y longitud que el campo apellido de la tabla empleados.

```
l_apellido empleados.apellido%TYPE;
```

El atributo %TYPE se utiliza para declarar una variable que hereda el tipo de dato de una columna, campo o variable específica. Es útil cuando se desea garantizar que la variable declarada tenga el mismo tipo de dato que una columna u otra variable, lo que facilita el mantenimiento del código y reduce la probabilidad de errores si por algún motivo ha cambiado el tipo de dato de la columna.

El atributo **%ROWTYPE** proporciona un tipo de registro que representa una fila en una tabla de base de datos. El registro puede almacenar una fila completa de datos seleccionados de la tabla o extraídos de un cursor o una variable de cursor. Los campos de un registro y las columnas correspondientes de una fila tienen los mismos nombres y tipos de datos.

```
variable tabla%ROWTYPE;
```

Puede usar el atributo %ROWTYPE en las declaraciones de variables como especificador de tipo de dato. Las variables declaradas con %ROWTYPE se tratan como aquellas declaradas con un nombre de tipo de dato.

%ROWTYPE es un atributo que permite declarar variables de tipo registro que reflejan la estructura exacta de una tabla. En nuestro caso la variable l_empleado es una variable que contiene todos los campos de la tabla empleados, éstos iguales en tipo y tamaño a los de la tabla referenciada.

```
l_empleado empleados %ROWTYPE;
```

El atributo %ROWTYPE se utiliza para declarar un registro que puede contener una fila completa de datos de una tabla o un cursor. Hereda la estructura (es decir, todas las columnas y sus tipos de datos correspondientes) de la tabla o el cursor. Resulta útil cuando se desea trabajar con una fila completa de datos y evitar declarar cada columna por separado. Asimismo, si la estructura de la tabla cambiara no es necesario cambiar el código de las aplicaciones.

Bloques Almacenados

Todos los bloques que se han mostrado hasta ahora son "anónimos"; no tienen nombre. Si usar bloques anónimos fuera la única forma de organizar las sentencias, sería muy difícil usar PL/SQL para crear una aplicación grande y compleja. En cambio, PL/SQL admite la definición de bloques de código con nombre, también conocidos como subprogramas. Los subprogramas pueden ser procedimientos o funciones.

Generalmente, un procedimiento se utiliza para realizar una acción y una función para calcular y devolver un valor.

Supongamos que necesito mostrar "Hola a Todos!" desde varios lugares de mi aplicación. Quiero evitar repetir la misma lógica en todos esos lugares. Y ¿qué sucede si necesito cambiar el mensaje, por ejemplo, a "Hola Alumnos!"? Tendría que encontrar todas las ubicaciones en mi código donde aparece esta lógica.

En su lugar, crearemos un procedimiento llamado **saludos** ejecutando el siguiente comando en lenguaje de definición de datos (DDL):

```
CREATE OR REPLACE PROCEDURE saludos IS
    l_mensaje VARCHAR2 (100) := Hola a Todos!;
BEGIN
    DBMS_OUTPUT.put_line (l_mensaje);
END saludos;
```

No es necesario que se coloque el nombre del bloque en la sentencia END final, pero ayuda a clarificar dónde termina el código. Además de llamar a programas creados por Oracle e instalados en la base de datos (como DBMS_OUTPUT.PUT_LINE), puedo llamar a mi propio subprograma dentro de un bloque PL/SQL:

```
BEGIN
    saludos;
END;
```

Al almacenar el procedimiento se han ocultado todos los detalles de cómo saludo dentro del cuerpo o sea, queda invisible la implementación del procedimiento. Ahora se puede llamar al procedimiento **saludos** y que muestre el mensaje deseado sin tener que escribir la llamada a DBMS_OUTPUT.PUT_LINE ni determinar la forma correcta de formatear la cadena. Puedo llamar a este procedimiento desde cualquier lugar de mi aplicación. Así, si alguna vez necesito cambiar esa cadena, lo haré en un solo lugar, el punto de definición de esa cadena.

El procedimiento **saludos** es muy simple. Los procedimientos reales contendrán mucho más código y casi siempre también tendrán parámetros. Los parámetros son datos que se pasan a los subprogramas al ser llamados y permiten crear subprogramas más flexibles y genéricos. Se pueden usar en muchos contextos diferentes.

Si quisiéramos mostrar "Hola Alumnos!" en lugar de "Hola a Todos!". Podría hacer una copia de mi procedimiento **saludos** y cambiar la cadena que muestra:

```
CREATE OR REPLACE PROCEDURE saludos2
IS
    l_mensaje VARCHAR2 (100) := 'Hola Alumnos!';
BEGIN
    DBMS_OUTPUT.put_line (l_mensaje);
END saludos2 ;
```

Sin embargo, podríamos terminar con docenas de variaciones del mismo procedimiento de **saludos**, lo que dificultaría enormemente el mantenimiento de mi aplicación. Un enfoque mucho mejor es analizar el procedimiento e identificar qué partes permanecen iguales (son estáticas) y cuáles cambian cuando el mensaje necesita cambiar. Puedo entonces pasar las partes cambiantes como parámetros y tener un único procedimiento que pueda usarse en diferentes circunstancias.

Así que cambiaremos el procedimiento **saludos** para tener un nuevo procedimiento y único para todos los casos que se necesite mandar un saludo a alguien:

```
CREATE OR REPLACE PROCEDURE saludos (a quien IN VARCHAR2) IS
    l_mensaje VARCHAR2 (100);
BEGIN
    l_mensaje := Hola ' || a quien;
    DBMS_OUTPUT.put_line (l_mensaje);
END saludos3;
```

Borraremos el procedimiento saludos2 siendo innecesaria su existencia en la Base de Datos:

```
DROP PROCEDURE Saludos2;
```

Después del nombre del procedimiento, se añade paréntesis de apertura y cierre, y dentro de ellos se proporciona el único parámetro de este código. Un procedimiento puede tener varios parámetros, pero cada uno sigue la misma estructura básica:

```
nombre_parámetro modo_parámetro tipo_datos
```

En otras palabras, debe proporcionarse un nombre para el parámetro, el modo o forma en que se utilizará (IN, sólo lectura) y el tipo de datos que se pasarán al subprograma a través de este parámetro.

En PL/SQL, los parámetros de los procedimientos pueden ser de tipo IN, OUT o IN OUT, permitiendo el intercambio de datos entre el procedimiento y el código que lo llama.

- IN (Entrada): El parámetro se utiliza para pasar datos al procedimiento, pero el procedimiento no puede modificar su valor.
- OUT (Salida): El parámetro se utiliza para que el procedimiento devuelva datos al código que lo llama.
- IN OUT (Entrada/Salida): El parámetro se utiliza tanto para pasar datos al procedimiento como para que el procedimiento devuelva datos.

En este caso, sólo se va a pasar una cadena de solo lectura al procedimiento **saludos**. Ahora se puede decir Hola a quien queramos sin tener que modificar el código, ni copiarlo:

```
BEGIN
    saludos ('Alumnos');
    saludos ('a Todos');
    saludos ('Universo');
END;
```

Ahora supongamos que no sólo se quiere mostrar los mensajes de "Hola". A veces necesito guardarlos en una tabla de base de datos; otras veces, debo devolver la cadena al entorno host para que se muestre en un navegador web. En otras palabras, necesito separar la forma en que se construye el mensaje de "Hola" de su uso (mostrado, guardado, enviado a otro programa, etc.). Puedo lograr este nivel de flexibilidad deseado integrando el código que construye el mensaje en su propia función:

```
CREATE OR REPLACE FUNCTION saludar (como IN VARCHAR2) RETURN VARCHAR2 IS
BEGIN
    RETURN 'Hola ' || como;
END saludar;
```

El tipo de programa ahora es FUNCIÓN (FUNCTION), y no PROCEDIMIENTO (PROCEDURE). El nombre del subprograma ahora describe los datos que se devuelven, no la acción que se realiza. El cuerpo o la implementación del subprograma ahora contiene una cláusula RETURN que construye el mensaje y lo devuelve al bloque que lo llama. La cláusula RETURN, después de la lista de parámetros, establece el tipo de datos que devuelve la función.

Con el código necesario para construir el mensaje dentro de la función **saludar**, se puede usar este mensaje de varias maneras. Por ejemplo, llamar a la función para recuperar el mensaje y asignarlo a una variable:

```
DECLARE
  l_mensaje VARCHAR2 (100);
BEGIN
  l_mensaje := saludar ('Universo');
END;
```

Tener en cuenta que se llamó a la función **saludar** como parte de una sentencia PL/SQL (en este caso, la asignación de una cadena a una variable). La función **saludar** devuelve una cadena, por lo que puede utilizarse en lugar de una cadena en cualquier sentencia ejecutable. También se puede reemplazar el código del procedimiento **saludos** para generar la cadena con una llamada a la función:

```
CREATE OR REPLACE PROCEDURE saludos (a quien IN VARCHAR2) IS
BEGIN
  DBMS_OUTPUT.put_line (saludar(a quien));
END saludos;
```

También puedo llamar a la función desde una sentencia SQL. En el siguiente bloque, por ejemplo, se insertará el mensaje en una tabla de base de datos:

```
BEGIN
  INSERT INTO tabla_mensaje (fecha_mensaje, texto_mensaje)
  VALUES (SYSDATE, saludar('Chicago'));
END;
```

Aunque la lógica de "saludar" y "saludos" sean muy simple, demuestra la eficacia de asignar nombres a una o más sentencias ejecutables (un algoritmo) y luego hacer referencia a dicho algoritmo simplemente especificando el nombre y los parámetros necesarios.

Los bloques PL/SQL con nombre permiten construir aplicaciones complejas que se pueden comprender y mantener con relativa facilidad.

CASE - Una alternativa útil a IF

La sentencia y expresión CASE ofrecen otra forma de implementar la ramificación condicional. Al usar CASE en lugar de IF, a menudo se pueden expresar las condiciones con mayor claridad e incluso simplificar el código. Existen dos formas de la sentencia CASE:

- **Sentencia CASE simple.** Esta asocia cada una o más secuencias de sentencias PL/SQL con un valor. La sentencia CASE simple elige qué secuencia de sentencias ejecutar, basándose en una expresión que devuelve uno de esos valores.
- **Sentencia CASE buscada.** Esta elige cuál de las secuencias de sentencias PL/SQL se ejecutará evaluando una lista de condiciones booleanas. Se ejecuta la secuencia de sentencias asociada con la primera condición que se evalúa como VERDADERA.

Además de las sentencias CASE, PL/SQL también admite expresiones CASE. Una expresión CASE tiene una forma similar a una sentencia CASE y permite elegir cuál de las expresiones evaluar. El resultado de una expresión CASE es un valor único, mientras que el resultado de una sentencia CASE es la ejecución de una secuencia de sentencias PL/SQL.

Sentencias CASE Simples

Una sentencia CASE simple permite elegir cuál de varias secuencias de sentencias PL/SQL se ejecutará, basándose en los resultados de una sola expresión. Las sentencias CASE simples tienen la siguiente forma:


```
CASE expresion
  WHEN resultado1 THEN
    sentencia1
  WHEN resultado2 THEN
    sentencia2
  ...
  ELSE
    sentencia_else
END CASE;
```

Tener en cuenta que la declaración CASE puede incluir una cláusula ELSE explícita, pero ésta es opcional. Si no se especifica explícitamente una cláusula ELSE propia, PL/SQL utiliza implícitamente lo siguiente:

```
ELSE
  RAISE CASE_NOT_FOUND;
```

A continuación se mostrará un ejemplo de una declaración CASE simple que utiliza el tipo de empleado como base para seleccionar el algoritmo de bonificación adecuado con ELSE explícito.

```
CASE l_tipo_empleado
  WHEN 'S' THEN premia_bono (l_id_emple);
  WHEN 'H' THEN premia_bono (l_id_emple);
  WHEN 'C' THEN premia_comision_bono (l_id_emple);
  ELSE
    RAISE invalido_tipo_empleado;
END CASE;
```

En otras palabras, si no se especifica una cláusula ELSE y ninguno de los resultados de las cláusulas WHEN coincide con el resultado de la expresión CASE, PL/SQL generará un error CASE_NOT_FOUND. Este comportamiento es diferente al de las sentencias IF. Cuando una sentencia IF carece de una cláusula ELSE, no ocurre nada si no se cumple la condición. Con CASE, la situación análoga genera un error.

Sentencias CASE de Búsqueda.

Una sentencia CASE de búsqueda evalúa una lista de expresiones booleanas y, cuando encuentra una expresión que evalúa como VERDADERA, ejecuta una secuencia de sentencias asociadas a esa expresión. Las sentencias CASE de búsqueda tienen la siguiente forma:

```
CASE
  WHEN expresion1 THEN
    sentencia1
  WHEN expresion2 THEN
    sentencia2
  WHEN expresion3 THEN
    sentencia3
  ...
  ELSE
    sentencia_else
END CASE;
```

La declaración CASE de búsqueda se ajusta perfectamente al problema de implementar mi lógica de bonificación, como se muestra a continuación

```
CASE
  WHEN l_salario BETWEEN 10000 AND 20000 THEN
    dar_bono(l_id_emple, 1500);
  WHEN salary > 20000 THEN
    dar_bono(l_id_emple, 1000);
  ELSE
    dar_bono(l_id_emple, 0);
END CASE;
```

Por cierto, dado que las cláusulas WHEN se evalúan en orden, se puede optimizar el código enumerando primero las cláusulas WHEN más probables. Además, si se tienen cláusulas WHEN con expresiones costosas (es decir, que requieren muchos ciclos de CPU y memoria), puede que convenga enumerarlas al final para minimizar las probabilidades de que se evalúen.

Es conveniente utilizar sentencias CASE de búsqueda cuando se desee usar expresiones booleanas como base para identificar un conjunto de sentencias a ejecutar. Al igual que utilizar sentencias CASE simples cuando se pueda basar esa decisión en el resultado de una sola expresión.

Uso de Expresiones CASE

Una expresión CASE devuelve un único valor, el resultado de la **expresión** elegida. Cada cláusula WHEN debe estar asociada a exactamente una expresión (no a una sentencia). No utilice punto y coma ni END CASE para marcar el final de la expresión CASE. Las expresiones CASE terminan con un simple END.

Una expresión CASE de búsqueda puede utilizarse para simplificar el código para aplicar bonificaciones. En lugar de escribir una declaración IF o CASE con tres llamadas diferentes a **dar_bono**, puedo llamar a **dar_bono** sólo una vez y usar una expresión CASE en lugar del segundo argumento:

```
dar_bono(l_id_emple,
CASE
  WHEN l_salario BETWEEN 10000 AND 20000 THEN 1500
  WHEN l_salario > 40000 THEN 500
  ELSE 0
END);
```

A diferencia de la sentencia CASE, no se genera ningún error si no se selecciona ninguna cláusula WHEN en una expresión CASE. En cambio, si no se cumple ninguna condición WHEN, la expresión CASE simplemente devolverá NULL.

Procesamiento Iterativo con Bucles

Los bucles en el código permiten ejecutar el mismo código más de una vez. Los bucles son un elemento muy común en programación, ya que gran parte de la actividad real modelada por nuestros programas implica procesamiento repetitivo.

Podríamos necesitar, por ejemplo, realizar una operación para cada mes del año anterior. Para citar un ejemplo común de la famosa tabla de empleados de Oracle, podríamos querer actualizar la información de todos los empleados de un departamento determinado.

PL/SQL ofrece tres tipos de bucles:

- El bucle FOR (numérico y de cursor)
- El bucle simple (o infinito)
- El bucle WHILE

Cada tipo de bucle está diseñado para un propósito específico con sus propios matices, reglas de uso y directrices para una construcción de alta calidad.

Para comprender cómo los diferentes bucles resuelven problemas de distintas maneras, consideremos los siguientes ejemplos. Para intentar ser más claro este apunte comenzaremos con ejemplos simples para luego dar algunas situaciones más complejas. En algunos de estos ejemplos, el procedimiento realiza una llamada a **mostrar_total_ventas** para los años específicos, para cada año entre los valores de los argumentos inicial y final (el procedimiento no está mostrado).

El bucle FOR

Oracle Database ofrece dos bucles FOR:

- un bucle FOR numérico y
- un bucle de cursor.

Bucle FOR Numérico

Con el **bucle FOR numérico**, se especifican los valores enteros inicial y final (separados por dos puntos, punto punto “..”), y PL/SQL realiza el resto del trabajo automáticamente, iterando sobre cada valor entero entre el inicio y el final y finalizando el bucle.

```
DECLARE
    inicio NUMBER := 1; -- Valor inicial del rango
    fin NUMBER := 5;    -- Valor final del rango
BEGIN
    FOR contador IN inicio .. fin LOOP
        -- Código a ejecutar en cada iteración
        DBMS_OUTPUT.PUT_LINE('El valor del contador es: ' || contador);
    END LOOP;
END;
/
```

Veamos una explicación de cada una de las líneas del código:

Sentencia	¿Qué hace?
inicio NUMBER := 1;	Se declara una variable inicio y se inicializa en 1.
fin NUMBER := 5;	Se declara una variable fin y se inicializa en 5.
FOR contador IN inicio .. fin LOOP	Este bucle itera sobre valores del contador desde inicio hasta fin.
DBMS_OUTPUT.PUT_LINE(...);	Imprime el valor actual de contador en cada iteración.
END LOOP;	Marca el final del bucle FOR.

Otro ejemplo utilizado dentro del procedimiento **mostrar_múltiples_años**:

```
PROCEDURE mostrar_múltiples_años (inicio IN PLS_INTEGER, fin IN PLS_INTEGER) IS
BEGIN
    FOR l_año_actual IN inicio .. fin
    LOOP
        mostrar_total_ventas (l_año_actual);
    END LOOP;
END mostrar_múltiples_años;
```

En el bucle FOR numérico, Oracle Database declara implícitamente el iterador (en los ejemplos anteriores: contador y l_año_actual), como un entero. No es necesario (ni se debe) declarar una variable con el mismo nombre. De hecho, si se declara dicha variable, el bucle FOR no la utilizará. Se trata, más específicamente, de una variable entera distinta a la declarada implícitamente por Oracle Database y utilizada en el cuerpo del bucle. Al no declararla, sólo puede usarse esa variable dentro del ciclo de repetición.

El bucle FOR del cursor tiene la misma estructura básica, pero con él se proporciona una forma de trabajar con los datos de una tabla en lugar del rango de enteros bajo/alto. Dejemos este último tipo de bucle para cuando veamos el concepto de **cursores**.

Si queremos que el contador se decremente en cada repetición, en lugar de incrementarse, debemos colocar la palabra **reverse** luego de **in** y antes del límite inferior; el contador comenzará por el valor del límite superior y finalizará al llegar al límite inferior decrementando de a uno. En el siguiente ejemplo se muestra la tabla de multiplicar del 3 desde el 5 hasta el cero:

```
BEGIN
  FOR i IN REVERSE 0..5 LOOP
    dbms_output.put_line('3 x ' || TO_CHAR(i) || ' = ' || TO_CHAR(i*3));
  END LOOP;
END;
/
```

Ciclos For anidados

Se pueden colocar un ciclo **for** dentro de otro ciclo **for**. Por ejemplo, con las siguientes líneas se muestran las tablas del 2 y del 5 del 1 al 10:

```
BEGIN
  FOR j IN 2..5 LOOP
    FOR i IN 1..10 LOOP
      dbms_output.put_line(TO_CHAR(j) || ' x ' || TO_CHAR(i) || ' = ' || TO_CHAR(j*i));
    END LOOP i;
    dbms_output.put_line(' ');
  END LOOP j;
END;
/
```

Bucle Simple (Bucle Infinito)

Se le llama simple por una razón: comienza con la palabra clave LOOP y termina con la sentencia END LOOP. El bucle terminará si se ejecuta EXIT, EXIT WHEN o RETURN dentro del cuerpo del bucle (o si se genera una excepción). Veamos un ejemplo de bucle simple:

```
PROCEDURE mostrar_multiples_anios (inicio IN PLS_INTEGER, fin IN PLS_INTEGER) IS
  l_anio_actual PLS_INTEGER := inicio;
BEGIN
  LOOP
    EXIT WHEN l_anio_actual > fin;
    mostrar_total_ventas (l_anio_actual);
    l_anio_actual:= l_anio_actual + 1;
  END LOOP;
END mostrar_multiples_anios;
```

El Bucle WHILE

El bucle WHILE es muy similar al bucle simple; pero una diferencia crucial es que verifica previamente la condición de terminación. Es decir, un bucle WHILE podría no ejecutar su cuerpo ni una sola vez.


```
PROCEDURE mostrar_múltiples_años (inicio IN PLS_INTEGER, fin IN PLS_INTEGER) IS
    l_año_actual PLS_INTEGER := inicio;
BEGIN
    WHILE (l_año_actual <= fin)
    LOOP
        mostrar_total_ventas (l_año_actual);
        l_año_actual:= l_año_actual + 1;
    END LOOP;
END mostrar_múltiples_años;
```

El bucle WHILE consta de una condición (una expresión booleana) y un cuerpo de bucle. Antes de cada iteración del cuerpo, Oracle Database evalúa la condición. Si se evalúa como VERDADERO, se ejecutará el cuerpo del bucle. Si se evalúa como FALSO o NULL, el bucle finalizará.

Tener presente que se debe incluir en código del cuerpo del bucle la sentencia que modifique la evaluación de la condición y, que en algún momento, provoque la detención del bucle. Caso contrario estaremos con un bucle infinito:

```
l_año_actual:= l_año_actual + 1;
```

Si un bucle WHILE no cambia de alguna manera la condición que se evalúa, éste nunca terminará.

Principio: Una entrada, Una salida

En todos los ejemplos de la sección anterior, el bucle FOR claramente requiere la menor cantidad de código. Generalmente, se busca la implementación más simple y legible para sus necesidades. Esto no significa que siempre se deba usar un bucle FOR.

Usar el bucle FOR es la mejor solución para el escenario descrito, ya que necesitaba ejecutar el cuerpo del bucle un número fijo de veces. En muchas otras situaciones, el número de veces que debe ejecutarse un bucle varía según el estado de los datos en la aplicación. También podría ser necesario terminar el bucle cuando se cumpla una condición determinada; en este caso, un bucle FOR no es una buena opción.

Un principio importante y fundamental en la programación estructurada es "una entrada, una salida", es decir, un programa debe tener un único punto de entrada y un único punto de salida. Un único punto de entrada no es un problema con PL/SQL; independientemente del tipo de bucle que se utilice, siempre hay un único punto de entrada al bucle: la primera sentencia ejecutable después de la palabra clave LOOP. Sin embargo, es muy posible construir bucles con múltiples rutas de salida. Se debe evitar esta práctica. Tener múltiples formas de terminar un bucle resulta en un código mucho más difícil de depurar y mantener de lo que sería de otro modo.

En particular, debe seguir estas dos pautas para la terminación de bucles:

1. No utilizar las sentencias EXIT ni EXIT WHEN dentro de bucles FOR y WHILE. Debe usarse un bucle FOR sólo cuando se desee iterar sobre todos los valores (enteros o registros) especificados en el rango. Una sentencia EXIT dentro de un bucle FOR interrumpe este proceso y subvierte la intención de esa estructura. Un bucle WHILE, por otro lado, especifica su condición de terminación en la propia sentencia WHILE.

El siguiente ejemplo presenta un bucle FOR con una salida condicional. Se desea mostrar las ventas totales de cada año dentro del rango especificado. Sin embargo, si alguna vez encuentra un año con cero ventas (calculado por la función total_ventas_por_año), un bucle FOR con una salida condicional:

```
PROCEDURE mostrar_multiples_anios (inicio IN PLS_INTEGER, fin IN PLS_INTEGER) IS
    l_anio_actual PLS_INTEGER := inicio;
BEGIN
    FOR l_anio_actual IN inicio .. fin
    LOOP
        mostrar_total_ventas (l_anio_actual);
        EXIT WHEN total_ventas_por_anio (l_anio_actual) = 0;
    END LOOP;
END mostrar_multiples_anios;
```

Con esto tendremos dos maneras de salir del bucle. En esta situación, escribamos el bucle FOR como un bucle WHILE, lo que significa que ahora también debo declarar el iterador y añadirle elementos dentro del bucle. Un bucle WHILE con una salida:

```
PROCEDURE mostrar_multiples_anios (inicio IN PLS_INTEGER, fin IN PLS_INTEGER)
IS
    l_anio_actual PLS_INTEGER := inicio;
BEGIN
    WHILE (l_anio_actual <= fin AND total_sales_for_year (l_current_year) > 0)
    LOOP
        mostrar_total_ventas (l_anio_actual);
        l_anio_actual:= l_anio_actual + 1;
    END LOOP;
END mostrar_multiples_anios;
```

No utilice las sentencias RETURN ni GOTO dentro de un bucle, ya que provocan la terminación prematura y no estructurada del bucle.

Veamos un ejemplo de un bucle FOR con un retorno. La función ventas_totales devuelve el total de ventas de los años especificados, pero si algún año tiene \$0 en ventas, la función debe finalizar el bucle y devolver el total de ventas actuales. Este ejemplo es un bucle FOR con dos instancias de RETURN:

```
FUNCTION ventas_totales ( inicio IN PLS_INTEGER, fin IN PLS_INTEGER) RETURN PLS_INTEGER
IS
    l_retorno PLS_INTEGER := 0;
BEGIN
    FOR l_anio_actual IN inicio .. fin
    LOOP
        IF total_ventas_por_anio (l_anio_actual) = 0 THEN
            RETURN l_retorno;
        ELSE
            l_retorno := l_retorno + total_ventas_por_anio (l_anio_actual);
        END IF;
    END LOOP;
    RETURN l_retorno ;
END ventas_totales;
```

Tener en cuenta que el bucle termina de dos maneras: iterando todos los enteros entre los años inicial y final o ejecutando RETURN dentro del bucle. Además, y en relación con esto, esta función ahora tiene dos instancias de RETURN, lo que significa que hay dos salidas de la función. Esta tampoco es una forma recomendada de diseñar funciones. Debe tener solo un RETURN en la sección ejecutable: la última línea de la función.

Se puede modificar la función anterior para que tanto el bucle como la función tengan una sola salida:

```
FUNCTION ventas_totales ( inicio IN PLS_INTEGER, fin IN PLS_INTEGER) RETURN PLS_INTEGER
IS
    l_retorno PLS_INTEGER := 0;
    l_anio_actual PLS_INTEGER := inicio;
BEGIN
    WHILE (l_anio_actua <= fin AND total_ventas_por_anio (l_anio_actual) > 0)
    LOOP
        l_retorno := l_return + total_sales_for_year (l_anio_actual);
        l_anio_actual:= l_anio_actual+ 1;
    END LOOP;
    RETURN l_retorno;
END ventas_totales;
```

Toda la lógica necesaria para terminar el bucle se encuentra ahora en la condición WHILE. Una vez finalizado, la función ejecuta un único RETURN para devolver el valor total de las ventas. Esta segunda implementación es ahora más sencilla y fácil de entender, y esto para un programa que ya es bastante simple. Al trabajar con algoritmos mucho más complejos, seguir las directrices de "una entrada, una salida" tendrá un impacto aún mayor en la legibilidad del código.

Conversión de Datos

En PL/SQL, la conversión de datos se maneja tanto explícita como implícitamente. La conversión implícita ocurre cuando PL/SQL puede convertir automáticamente un tipo de dato en otro sin que el desarrollador especifique ninguna función. La conversión explícita se realiza usando funciones como CAST y TO_CHAR para forzar la conversión de datos a un tipo específico.

Conversión Implícita

- PL/SQL, en algunos casos, permite la conversión implícita, como cuando se suma un valor de tipo NUMBER a una variable VARCHAR2 que contiene un valor numérico.

```
DECLARE
    num1 NUMBER(3):=34;
    texto VARCHAR2(3):='22';
    num2 NUMBER(3);
BEGIN
    num2:=num1 + texto;
    dbms_output.put_line(num2);
END;
/
```

Conversión Explícita

- Función CAST(): Permite convertir datos de un tipo a otro.

CAST(fecha_texto AS DATE) (Convierte el valor fecha_texto de tipo texto a tipo fecha).

- Función TO_CHAR(): Permite convertir datos a tipo VARCHAR2 (texto).

TO_CHAR(fecha, 'YYYY-MM-DD') (Convierte la fecha al formato "AAAA-MM-DD").

- Función TO_NUMBER(): Permite convertir texto a números.

TO_NUMBER('123.45') (Convierte la cadena '123.45' a tipo NUMBER).

Ejemplo:

Supongamos que tienes una variable llamada v_numero de tipo NUMBER y quieres mostrarla en pantalla con dos decimales.

```
DECLARE
  v_numero NUMBER := 123.456;
BEGIN
  DBMS_OUTPUT.PUT_LINE('Número original: ' || v_numero); -- Imprime 123,456
  DBMS_OUTPUT.PUT_LINE('Número formateado: ' || TO_CHAR(v_numero, '99999.99'));
  -- Imprime 123.46
END;
/
```

Mostrará:

```
Número original: 123,456
Número formateado: 123.46
```

Ejercicios de Bloques Almacenados

1. Crear un bloque PL/SQL que muestre un comentario acorde a la nota que ha sacado un alumno: 10 – Sobresaliente, 9 – Excelente, 8 – Muy Bueno, 7 – Bueno, 6 – Aprobado, debajo de 6 Reprobado. (este ejercicio es igual al de TP1 pero ahora lo resolveremos con CASE)
2. Escribir los 10 primeros números pares de menor a mayor.
3. Escribir los 10 primeros números impares de mayor a menor.
4. Escribir un procedimiento que muestre la tabla de multiplicar deseada (del 1 al 12). Crear el bloque anónimo para probarlo
5. Armar un bloque que escriba en pantalla la cadena 'BASE DE DATOS II' de atrás para adelante ('IISOTAD ED ESAB').
6. Armar un procedimiento que escriba los 10 primeros números pares ya sea de menor a mayor (ascendente) o de mayor a menor (descendente). Crear el bloque anónimo para probarlo.
7. Crear una función PL/SQL que al ingresar una fecha devuelva cuál es el trimestre del año al cual pertenece. Crear el bloque anónimo para probarla.
8. Crear una función que dada una frase indique cuántas palabras la forman. Hay que considerar que las palabras se separan por espacio, no considerar los espacios finales de la frase dada.
9. Armar un bloque que al pedir una palabra y le dé vuelta sólo si es una palabra. En caso de que sea una frase (contenga uno o más espacios en el medio, no al final) indicar que no se cumple con la consigna dada.
10. Armar una función que dada la fecha de nacimiento de una persona indique cuál es su edad. Crear el bloque anónimo para probarla.